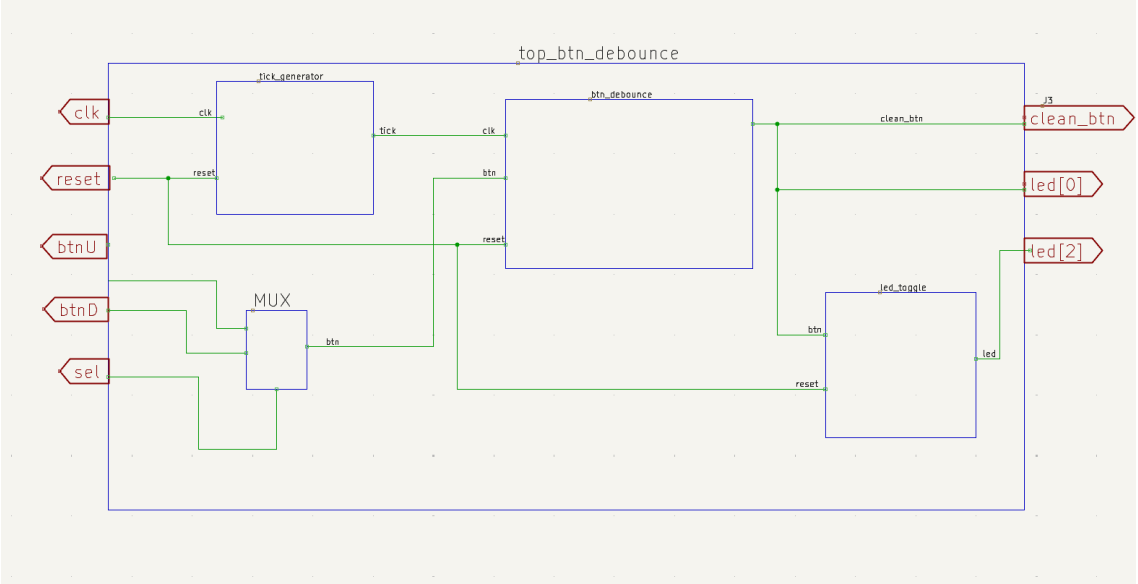

top_btn_debounce
tick_generator
btn_debounce
clk
reset
btnU
btnD
sel
MUX
tick
btn
clean_btn
led_toggle
led
led[0]
led[2]


Name
Value
clk
reset
btnU
btnD
sel
clean_btn
led[2:0]
CLK_FREQ[31:0]
CLK_PERIOD[31:0]
SIMULATION_DIV[31:0]
SAMPLE_PERIOD[31:0]
BTN_PRESS_LIMIT[31:0]
000
101
100

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
//////////
// Company:
// Engineer:
//
// Create Date: 2026/10/16 13:00:00
// Design Name:
// Module Name: tb_top_btn_debounce
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
//////////
module tb_btn_debounce();
    parameter CLK_FREQ =100000000; // 100MHz
    parameter CLK_PERIOD =10;      // 10ns (100MHz 1주기)
    // 시뮬레이션 시간 단축을 위한 분주비 오버라이드 값
    // 실제 tick_generator 분주비 : 100,000
    // 시뮬레이션용 분주비 : 100 (100배 단축)
    parameter SIMULATION_DIV =100;
    localparam SAMPLE_PERIOD =SIMULATION_DIV *CLK_PERIOD; // 1000ns = 1us
    // 디바운스 완료(10틱)를 확실히 넘기 위해 충분히 유지 (15us)
    parameter BTN_PRESS_LIMIT =SAMPLE_PERIOD *15;
    // tick_generator의 카운터 한계값을 시뮬레이션용 값으로 덮어 씀
    defparam u_top_btn_debounce.u_tick_generator.DIV =SIMULATION_DIV;
    // 테스트벤치용 신호 선언
    reg clk;
    reg reset;
    reg btnU;
    reg btnD;
```

```
    reg sel;

    wire clean_btn;
    wire [2:0] led;
    top_btn_debounce u_top_btn_debounce (
        .clk(clk),
        .reset(reset),
        .btnU(btnU),
        .btnD(btnD),
        .sel(sel),
        .clean_btn(clean_btn),
        .led(led)
    );
    // 100MHz 클럭 생성
    initial begin
        clk =0;
        forever begin
            #(CLK_PERIOD /2);
            clk =~clk;
        end
    end
    initial begin
        reset =1;
        btnU  =0;
        btnD  =0;
        sel   =0;
        #200;
        reset =0;
        #500;
        $display("[%0t] === START sel=0, btnU Test ===", $time);

        btnU =1; #200;
        btnU =0; #300;
        btnU =1; #120;
        btnU =0; #500;
        btnU =1;
        #(BTN_PRESS_LIMIT);
        #1000;
        btnU =0;
```

```
        #(SAMPLE_PERIOD *15);
        #2000;
        $display("[%0t] ---- ALL SIM TESTS FINISHED ----", $time);
        $finish;
    end
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/16 11:59:18
// Design Name:
// Module Name: top_btn_debounce
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module top_btn_debounce(
    input wire clk,
    input wire reset,
    input wire btnU,
    input wire btnD,
    input wire sel,
    output wire clean_btn,
    output wire [2:0] led
);
    wire w_tick;
    wire w_btn;
    tick_generator u_tick_generator (
        .clk(clk),
        .reset(reset),
        .tick(w_tick)
    );
    mux u_mux (
```

```
        .sel(sel),
        .btnU(btnU),
        .btnD(btnD),
        .btn(w_btn)
    );
    button_debounce u_button_debounce (
        .clk(w_tick),
        .reset(reset),
        .btn(w_btn),
        .clean_btn(clean_btn)
    );
    led_toggle u_led_toggle (
        .reset(reset),
        .btn(clean_btn),
        .led(led[2])
    );
    assign led[0] =clean_btn;
    assign led[1] =1'b0;
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/16 11:50:05
// Design Name:
// Module Name: led_toggle
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module led_toggle(
    input wire reset,
    input wire btn,
    output wire led
);
    reg r_toggle =1'b0;
    always @(posedge btn or posedge reset) begin
        if (reset) begin
            r_toggle <=1'b0;
        end else begin
            r_toggle <=~r_toggle;
        end
    end
    assign led =r_toggle;
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/15 14:19:12
// Design Name:
// Module Name: button_debounce
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module button_debounce(
    input wire clk,
    input wire reset,
    input wire btn,
    output reg clean_btn
    );
     reg [$clog2(10)-1:0] r_count;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_count   <=0;
            clean_btn <=0;
        end
        else if (r_count ==10 -1) begin
            r_count   <=0;
            clean_btn <=btn;
        end
        else begin
```

```
            r_count <=r_count +1;
        end
    end
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/15 15:55:16
// Design Name:
// Module Name: tick_generator
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module tick_generator #(
    parameter DIV =100000
)(
    input wire clk,
    input wire reset,
    output reg tick
    );
    reg [$clog2(DIV)-1:0] r_counter =0;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_counter <=0;
            tick  <=1'b0;
        end
        else begin
            if (r_counter == (DIV -1)) begin
                r_counter <=0;
                tick  <=1'b1;
```

```
            end
            else begin
                r_counter <=r_counter +1;
                tick  <=1'b0;
            end
        end
    end
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/16 11:57:50
// Design Name:
// Module Name: mux
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module mux(
    input wire sel,
    input wire btnU,
    input wire btnD,
    output wire btn
);
    assign btn = (sel) ? btnD : btnU;
endmodule
```